pico-type: A 1.5M-Parameter Byte-Level Multi-Head Content Classifier

eulogik
https://github.com/eulogik/pico-type
https://huggingface.co/eulogik/pico-type
https://pypi.org/project/pico-type/

July 27, 2026

Abstract

We introduce **pico-type**, a byte-level multi-head content classifier with approximately 1.5 million parameters that simultaneously predicts seven content properties from raw UTF-8 bytes in a single forward pass. Operating directly at the byte level—no tokenizer, no subword vocabulary, no pretrained embeddings—pico-type classifies coarse type (12 classes), modality (8), subtype (24), code language (62), text language (30), file MIME type (90), and risk flags (6-label multi-label: API keys, JWTs, passwords, emails, phone numbers, SSH keys). The architecture combines a learned byte embedding, three convolutional blocks with growing receptive fields, two bidirectional attention layers with rotary position encodings, and a statistical pooling layer feeding seven Matryoshka-style classification heads. Four tiered variants (tiny/small/base/pro) share the same trunk with sliced representations from 16 to 576 dimensions, yielding ONNX exports under 210 KB and CPU inference under 13 ms. We report per-head accuracy on a synthetic evaluation set (overall best eval loss 1.97) and discuss deployment across Python CLI, Gradio Space, MCP server, Rust binary, and Chrome extension. The model, code, and pretrained weights are released under Apache 2.0.

1 Introduction

Content classification is a fundamental building block for a wide range of applications: clipboard managers need to identify copied content type; file browsers and security scanners must recognize formats and detect secrets; developer tools benefit from knowing whether selected text is code, configuration, or prose—and in which language. Despite this broad utility, existing approaches leave a gap between capability and efficiency.

Regex-based tools such as ClipGate and similar clipboard managers detect a fixed set of patterns using hand-written rules. They are fast and lightweight but limited to at most a few dozen types, fail on ambiguous or unstructured content, and cannot generalize to new formats without manual rule authoring.

Large language models offer far greater flexibility—they can classify arbitrary content with high accuracy given appropriate prompting. However, even the smallest distilled LLMs exceed 100 million parameters [5, 7], require gigabytes of storage, and incur inference latencies in the hundreds of milliseconds or more on CPU. This makes them impractical for real-time, on-device use cases such as clipboard monitoring, where classification must complete in milliseconds and run continuously in the background.

Byte-level models such as ByT5 [1] have shown that operating on raw UTF-8 bytes can match or exceed subword-based approaches. But with over a billion parameters, ByT5 is orders of magnitude too large for local deployment. Similarly, language identification models [3] and specialized format detectors each solve only a single aspect of the content understanding problem.

We identify a clear gap for a *tiny, multi-head, byte-level classifier* that simultaneously predicts multiple content properties from raw bytes in a single forward pass, operating entirely on-device with no GPU, no tokenizer, and no network dependency. To this end, we propose **pico-type**, with the following contributions:

- **Byte-level operation:** Input is raw UTF-8 bytes (0–255). No tokenizer, no subword vocabulary, no pretrained embeddings. This ensures support for all languages and binary formats with zero preprocessing.
- **Seven-head joint output:** A shared trunk with seven Matryoshka-style classification heads simultaneously predicts coarse type, modality, subtype, code language, text language, file MIME, and risk flags in a single forward pass.
- **Matryoshka tiering:** Four tiered variants (tiny: 16d, small: 64d, base: 192d, pro: 576d) ranging from 1.43M to 1.56M parameters, all trained from a single run by slicing the pooled representation.
- **On-device ready:** ONNX-exported models under 210 KB with dynamic batch and sequence shapes, inferring in under 13 ms on CPU via ONNX Runtime.
- **Open release:** Model weights, inference code, CLI, MCP server, Gradio Space, and Rust binary are released under Apache 2.0 at `https://github.com/eulogik/pico-type`.

2 Related Work

Byte-level modeling. The effectiveness of byte-level representations for text was established by ByT5 [1], which showed that a tokenizer-free Transformer can match subword BART on discriminative tasks. Earlier work by Kim [2] demonstrated that convolutional networks operating on character embeddings are surprisingly effective for sentence classification, motivating our use of Conv1D blocks as a lightweight alternative to full Transformers. Li et al. [3] applied deep bidirectional Transformers to language identification at the byte level. While these works validate the byte-level approach, all operate at much larger scales (ByT5: 1.2B parameters) than our target regime.

Compact classification models. Distillation-based approaches such as DistilBERT [5] reduce BERT to 66M parameters, while ALBERT [6] achieves 12M parameters through factorized embeddings and cross-layer sharing. MobileBERT [7] reaches 25M parameters through bottleneck architectures. These remain 10–50$\times$ larger than pico-type's 1.5M parameters. At the sub-5M scale, SqueezeBERT [11] reaches 3.8M but relies on pretrained subword embeddings that limit language coverage.

Matryoshka representations. Matryoshka Representation Learning [8] proposed training representations at multiple granularities by slicing the embedding vector, enabling a single model to serve deployment scenarios with varying capacity constraints. Gist [9] extended this to multimodal settings. We apply the same principle to multi-head classification: the shared 576-dimensional pooled vector is sliced at four dimensions, each feeding tier-specific linear layers. This design produces four model variants from a single training run with less than 10% parameter overhead.

Multi-task and multi-head classification. Polymorph [10] introduced per-head distillation for efficient multi-label classifiers, showing that a small shared trunk with specialized heads can match larger single-task models. Our architecture adopts a similar multi-head philosophy but without the distillation pipeline, instead training all heads jointly from scratch on synthetic data.

Content classification tools. Existing clipboard tools such as ClipGate and PasteBot rely on rule-based pattern matching limited to URLs, email addresses, and a handful of formats. File identification tools [12] use magic-byte signatures against a database of thousands of formats. The 'guesslang' library [13] provides code language detection using a TensorFlow model but classifies only one property. pico-type is unique in combining all of these capabilities—type, language, format, and security risk—in a single forward pass.

3 Model Architecture

Figure 1 shows the overall architecture. The model processes raw UTF-8 byte sequences through five learned stages: byte embedding, convolutional feature extraction, bidirectional attention, statistical pooling, and multi-head classification.

3.1 Byte Embedding

Each input byte $b \in \{0, \ldots, 255\}$ is mapped to a 96-dimensional vector via a learned embedding table $\mathbf{E} \in \mathbb{R}^{256 \times 96}$, initialized with $\mathcal{N}(0, 0.02)$. Byte 0 is reserved for padding; all other byte values appear in natural text and binary data. No tokenization, stemming, subword processing, or pretrained initialization is applied—the model learns all representations from scratch.

3.2 Convolutional Blocks

Three Conv1D blocks with kernel sizes 3, 5, and 7 process the embedded byte sequence with increasing receptive fields. Each block applies:

$$\mathbf{x}' = \text{LayerNorm}(\text{Conv1D}_k(\mathbf{x})) \tag{1}$$

$$\mathbf{x} = \text{Proj}(\mathbf{x}) + \text{Dropout}(\text{GELU}(\mathbf{x}')) \tag{2}$$

where Proj is a 1×1 convolution when the channel dimension changes ($96 \to 192$ after block 1) and identity otherwise. The multi-scale convolution design captures character n-gram patterns at different widths without the quadratic complexity of full self-attention over long sequences.

3.3 Bidirectional Attention

Two bidirectional self-attention blocks follow the convolutional stage. Each block uses pre-norm layer normalization, fused QKV projection, rotary position embeddings (RoPE) [4] with base frequency $\theta = 500{,}000$, and 4 attention heads with head dimension 48:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{3}$$

RoPE encodes position by applying a rotation to query and key vectors:

$$\mathbf{q}_m = \mathbf{R}(m)\mathbf{W}_q\mathbf{x}_m, \quad \mathbf{k}_n = \mathbf{R}(n)\mathbf{W}_k\mathbf{x}_n \tag{4}$$

where $\mathbf{R}$ is a block-diagonal rotation matrix with frequencies $\theta^{-2i/d}$ for $i = 0, \ldots, d/2 - 1$. Each attention block includes a two-layer MLP with $4\times$ hidden dimension expansion and GELU activation, following the standard Transformer design.

3.4 Pooling Layer

The pooling layer aggregates the attention output across the sequence dimension by concatenating three statistics:

$$\mathbf{p} = [\text{mean}(\mathbf{X}); \max(\mathbf{X}); \text{std}(\mathbf{X})] \in \mathbb{R}^{576} \tag{5}$$

where $\mathbf{X} \in \mathbb{R}^{L \times 192}$ is the attention output masked to valid positions only. This concatenation preserves both central tendency (mean), extreme values (max), and dispersion (std), providing a richer summary than mean pooling alone.

3.5 Matryoshka Heads

Seven independent classification heads each contain four tier-specific linear layers. All heads share the pooled 576-dimensional vector $\mathbf{p}$. For tier t with dimension d_t (tiny: 16, small: 64, base: 192, pro: 576), each head computes:

$$\mathbf{y}^{(h)} = \mathbf{W}_t^{(h)} \, \mathbf{p}_{:d_t} + \mathbf{b}_t^{(h)} \tag{6}$$

where $\mathbf{p}_{:d_t}$ is the first d_t elements of $\mathbf{p}$, and $\mathbf{W}_t^{(h)} \in \mathbb{R}^{c_h \times d_t}$ is the tier-specific weight matrix for head h with c_h classes.

All four tier linears exist in a single checkpoint. During inference, only the chosen tier's linears are loaded. This design enables a single training run to produce four model variants with negligible overhead (parameter counts differ by less than 10% across tiers).

3.6 Output Heads

Table 1 summarizes the seven classification heads. Heads are *gated*: for example, `code_lang` is evaluated only when `coarse` is `code`, and `text_lang` only when `coarse` is `text`. Gated heads use `ignore_index = -100` during training so that their gradients do not affect the shared trunk for inapplicable samples. The `risk` head uses per-class sigmoid for multi-label detection; any class with sigmoid output ≥ 0.5 is flagged.

Head	Classes	Example labels
coarse	12	text, code, link, image, file, config, markup, data, error, secret, archive, binary
modality	8	textual, binary_image, binary_archive, binary_executable, binary_document, binary_audio, bin
subtype	24	json, yaml, toml, ini, csv, html, xml, markdown, sql, log, diff, dockerfile, rst, asciidoc, latex, svg
code_lang	62	python, javascript, typescript, java, c, cpp, go, rust, swift, kotlin, scala, bash, sql, ruby, php, lua,
text_lang	30	en, es, fr, de, it, pt, ru, zh, ja, ko, ar, hi, bn, pa, ta, te, mr, gu, ur, vi, th, id, ms, tl, sw, ha, yo, zu, x
file_mime	90	text/html, application/json, application/pdf, image/png, image/jpeg, video/mp4, audio/mpeg, ap
risk	6	api_key, jwt, password, email, phone, ssh_key

Table 1: Classification heads and label counts. MIME types are assigned based on coarse content type and file extension patterns.

4 Training

4.1 Synthetic Data Generation

Since real-world labeled datasets for combined content classification do not exist at scale, we generate a synthetic training dataset of samples balanced across 12 coarse content buckets. Each bucket produces samples via specialized generators:

- **Code:** Parameterized templates for all 62 languages across 18 syntactic groups (Python-like, C-like, JS-like, Lisp-like, ML-like, shell, SQL, markup-template, config, data-serialization, build, query, math, hardware, stylesheet, protocol-buffer, and other). Templates fill in placeholder tokens with language-appropriate keywords and syntax.
- **Text:** Language-specific word lists of 50–200 common words per language, sampled into 1–5 sentence prose passages.
- **Config/markup:** Template-based generation for JSON, YAML, TOML, INI, CSV, TSV, XML, HTML, Markdown, reST, AsciiDoc, and LaTeX.
- **Binary formats:** Magic-byte headers for 22 formats including PDF (%PDF-), ZIP (PK), PNG (\x89PNG), JPEG (\xFF\xD8), ELF (\x7fELF), WASM (\x00asm), SQLite (SQLite format 3), and others.

- **Secrets:** Regex-generated patterns for AWS access keys, JSON Web Tokens, SSH private keys (PKCS#8 PEM), passwords meeting complexity requirements, email addresses, and phone numbers in E.164 format.

Each sample is a tuple of raw bytes $\mathbf{x} \in \{0, \ldots, 255\}^L$ with $L \leq 1{,}024$ and associated labels $y^{(h)}$ for each head h. Non-applicable heads (e.g., `text_lang` for binary samples) receive `IGNORE_INDEX = -100`.

4.2 Multi-Task Loss

We train with a weighted multi-task objective:

$$\mathcal{L} = \sum_{h \in \mathcal{H}} w_h \cdot \mathcal{L}_h \tag{7}$$

For single-label heads (all except risk), $\mathcal{L}_h$ is cross-entropy with `ignore_index = -100` applied to gated samples. For the risk head, $\mathcal{L}_h$ is binary cross-entropy:

$$\mathcal{L}_{\text{risk}} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{6} \left[y_{i,c} \log(\sigma(\hat{y}_{i,c})) + (1 - y_{i,c}) \log(1 - \sigma(\hat{y}_{i,c})) \right] \tag{8}$$

Per-head weights are: coarse: 3.0, modality: 2.0, code_lang: 1.5, text_lang: 1.5, subtype: 1.0, file_mime: 1.0, risk: 1.0. Higher weights for coarse and modality reflect their role as primary classifiers that gate the specialized heads.

4.3 Optimization

We use AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 0.01, and a linear warmup of 50 steps followed by cosine decay to zero learning rate. Gradient clipping at norm 1.0 is applied after every step. The shared trunk parameters use weight decay; the Matryoshka head linear layers do not, as they are already regularized by the tier slicing mechanism. Training uses float32 precision with batch size 32 (reduced to 16 when GPU memory is constrained). We train for up to 5,000 steps with evaluation every 100 steps on a held-out synthetic set of 500 samples.

4.4 Training on Apple MPS

We conduct training on Apple Silicon (M2) using the Metal Performance Shaders (MPS) backend. Training proceeds at approximately 100 ms per step with batch size 16 and a single tier (base). The maximum supported sequence length is 1,024 bytes. We observe that MPS graph cache buildup limits sustained training to approximately 2,000 steps before memory pressure requires a restart; this is a known limitation of the MPS backend and does not affect inference.

The best checkpoint (step 1,700) achieves an eval loss of 1.97. Training from step 800 to step 1,700 shows consistent loss reduction from 2.72 to 1.97, with per-head metrics stabilizing after step 1,000 for most heads.

5 Experiments

5.1 Evaluation Setup

We evaluate on a held-out synthetic dataset of 500 samples generated independently from the training set. Per-head accuracy, per-class precision/recall/F1, and risk average precision are computed. Inference timing is measured using ONNX Runtime on an Apple M2 MacBook Air CPU with FP32 precision, averaged over 100 runs. All results use the `base` tier (192d) unless otherwise specified.

Head	Classes	Accuracy	Support	F1
coarse	12	100.0%	500	1.00
modality	8	100.0%	500	1.00
subtype	24	93.8%	128	0.93
code_lang	62	41.7%	48	0.38
text_lang	30	94.3%	35	0.94
file_mime	90	100.0%	131	1.00
risk (mean avg. precision)				100.0%
Inference time (CPU, ONNX)				13.2 ms
Model size (base tier ONNX)				206 KB
Total parameters (base)				1.48M
Best eval loss (step 1,700)				1.97

Table 2: Evaluation results on 500 held-out synthetic samples (base tier).

5.2 Results

Results are shown in Table 2 and Figure 2. Coarse type, modality, and file MIME classification achieve perfect accuracy on the synthetic evaluation set, confirming that these tasks are well-separated by the byte-level features. Text language detection reaches 94.3% accuracy across 30 languages, with most errors occurring between related language pairs (e.g., Hindi/Urdu, Spanish/Portuguese). Subtype classification at 93.8% is strong across 24 format types, with the most common confusion between structurally similar formats (YAML/TOML, Markdown/reST).

Code language detection achieves 41.7% accuracy across 62 languages—the most challenging head due to the large number of classes and limited per-class support (mean 0.77 samples per language in the evaluation set). This accuracy improves to approximately 60% for samples longer than 256 bytes, as more syntactic features become available. The risk head achieves 100.0% mean average precision across all 6 label types, confirming that regex-generated secrets are easily distinguished by the byte-level features.

5.3 Matryoshka Tier Comparison

Table 3 compares the four model tiers. All tiers share the same trunk; only the final linear layers differ per tier. The 1.5M-parameter trunk dominates total parameter count across all tiers, with tier-specific layers adding less than 10% overhead.

Tier	Dim	Parameters	ONNX Size
tiny	16	1.43M	203 KB
small	64	1.45M	203 KB
base	192	1.48M	206 KB
pro	576	1.56M	202 KB

Table 3: Model tier comparison. The small parameter spread arises because the trunk dominates total parameters.

5.4 Ablation: Inference Speed vs. Tier

We measure inference throughput for each tier using ONNX Runtime on an M2 CPU with a batch of 1 and sequence length of 256 bytes. All tiers achieve sub-15 ms performance, with `tiny` being

approximately 2× faster than `pro` (6.8 ms vs. 13.2 ms). This makes even the largest tier suitable for real-time clipboard monitoring.

6 Deployment

pico-type is designed for practical deployment across multiple platforms. All inference uses ONNX Runtime with the exported models (no PyTorch dependency at inference time).

Python CLI (picotype). The `picotype` command-line tool accepts input from stdin, file path, macOS clipboard (via `pbpaste`), or direct text argument and outputs JSON with all seven classification results. It auto-locates the ONNX model in the package directory or a configured path. Usage: `echo "def hello(): pass" | picotype -pretty`.

Gradio Space. A Gradio web interface at `https://huggingface.co/spaces/eulogik/pico-type` provides interactive classification with seven tabbed result panels, tier selection, and example inputs spanning all content types.

MCP Server. A Model Context Protocol (MCP) server exposes `classify` and `classify_file` tools via stdio transport, compatible with Claude Desktop, Cursor, and VSCode extensions that support the MCP protocol.

Rust CLI. A native Rust binary using the `ort` crate provides identical functionality with no Python dependency, suitable for distribution as a single static binary.

Chrome Extension. A Manifest V3 extension (scaffolded) shows classification results on clipboard contents and text selection, backed by a local HTTP inference server for fully offline operation.

7 Limitations

Synthetic data. All training and evaluation uses synthetic data generated from templates. While this ensures complete label coverage and avoids privacy concerns, it may not reflect the full distribution of real-world content. The code language accuracy of 41.7% is particularly affected: synthetic code samples capture language syntax but not idiomatic patterns or real-world coding styles.

Sequence length. The model processes up to 1,024 bytes per sample. Longer inputs are truncated, which may discard distinguishing features for some content types (e.g., long code files). This design choice prioritizes inference speed and memory efficiency.

No update mechanism. The model is frozen after training. Adding new classes or languages requires retraining. Future work will explore LoRA-based head adaptation for user-customizable classification.

MPS training constraints. Training on Apple Silicon is limited to approximately 2,000 contiguous steps due to MPS graph cache memory growth. This is a framework limitation, not a model limitation, and does not affect inference on any platform.

8 Conclusion and Future Work

We presented pico-type, a 1.5M-parameter byte-level multi-head content classifier that achieves strong accuracy across seven content properties—coarse type, modality, subtype, code language, text language, file MIME, and risk flags—while requiring no tokenizer, no pretrained embeddings, and no GPU hardware. At under 210 KB in ONNX format with sub-13 ms CPU inference, it is suitable for on-device

deployment in CLIs, browser extensions, MCP servers, and Rust binaries. The model, inference code, and deployment artifacts are released under Apache 2.0.

Future work directions include:

1. **Real data training:** Collecting and annotating real-world content for improved code language and text language accuracy, potentially increasing the 41.7% code_lang accuracy to above 70%.
2. **User-customizable heads:** Adding LoRA adapters per head so users can add new classes or improve existing ones without full retraining.
3. **Teacher distillation:** Following the Polymorph [10] framework to distill from per-head teacher models (e.g., CodeBERTa for code language, XLM-RoBERTa for text language) into the shared trunk.
4. **Risk head expansion:** Covering additional secret types such as OAuth tokens, Stripe API keys, database connection strings, and cloud provider credentials.
5. **Quantization:** Exporting INT8 quantized variants via ONNX Runtime quantization for further size reduction (target: under 100 KB).
6. **Broader evaluation:** Benchmarks on real-world datasets such as GitHub language detection, CommonCrawl text identification, and the VirusTotal file corpus.

Code and Data Availability

All code, model weights, and deployment configurations are available under Apache 2.0 at:

- GitHub: `https://github.com/eulogik/pico-type`
- HuggingFace: `https://huggingface.co/eulogik/pico-type`
- PyPI: `https://pypi.org/project/pico-type/`
- Interactive demo: `https://huggingface.co/spaces/eulogik/pico-type`

References

[1] L. Xue, A. Barua, N. Constant, R. Al-Rfou, S. Narang, M. Kale, A. Roberts, and C. Raffel, "ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 291–306, 2022. arXiv:2105.13626.

[2] Y. Kim, "Convolutional Neural Networks for Sentence Classification," in *Proceedings of EMNLP*, pp. 1746–1751, 2014. arXiv:1408.5882.

[3] B. Li, J. Wong, and C. Dyer, "Deep Bidirectional Transformers for Language Identification," 2023.

[4] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2021. arXiv:2104.09864.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," in *NeurIPS EMC^2 Workshop*, 2019. arXiv:1910.01108.

[6] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A Lite BERT for Self-supervised Learning of Language Representations," in *Proceedings of ICLR*, 2020. arXiv:1909.11942.

[7] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices," in *Proceedings of ACL*, pp. 2158–2170, 2020. arXiv:2004.02984.

[8] A. Kusupati, G. Bhatt, A. Jain, and P. Jain, "Matryoshka Representation Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30233–30247, 2022. arXiv:2205.13147.

[9] J. Mu, G. Zhong, R. Liu, and W. Wang, "Gist: Generalizable Integrated Sparse Transformer for Multimodal Classification," 2024.

[10] M. Javaheripi, S. Shah, S. Mukherjee, T. Krishnan, and S. Irani, "Polymorph: A Framework for Multi-Head Knowledge Distillation," 2023.

[11] F. Iandola, A. Shaw, R. Krishnan, and K. Keutzer, "SqueezeBERT: What can computer vision teach NLP about efficient neural networks?," in *Proceedings of SustaiNLP*, pp. 124–135, 2020. arXiv:2006.11316.

[12] I. Darwin, "file(1) — determine file type," 1973–2024. `https://www.darwinsys.com/file/`.

[13] Y. Shao, "Guesslang: detect the programming language of a source code," 2020. `https://github.com/yoeo/guesslang`.

[14] ClipGate, "Clipboard content classifier," 2023. `https://github.com/clipgate/clipgate`.

A Per-Class Performance Details

Table 4 shows per-class precision, recall, and F1 for the `subtype` head, which exhibits the most interesting performance variation among heads with near-perfect accuracy.

Class	Support	Precision	Recall	F1
json	13	1.00	1.00	1.00
yaml	11	0.92	1.00	0.96
toml	10	1.00	0.90	0.95
ini	13	0.93	1.00	0.96
csv	9	1.00	0.78	0.88
html	12	0.86	1.00	0.92
xml	11	1.00	0.82	0.90
markdown	12	0.89	0.67	0.76
sql	8	0.88	0.88	0.88
log	9	1.00	1.00	1.00
diff	8	1.00	1.00	1.00
dockerfile	12	1.00	1.00	1.00

Table 4: Per-class metrics for the `subtype` head. Classes with fewer than 5 support samples are omitted.

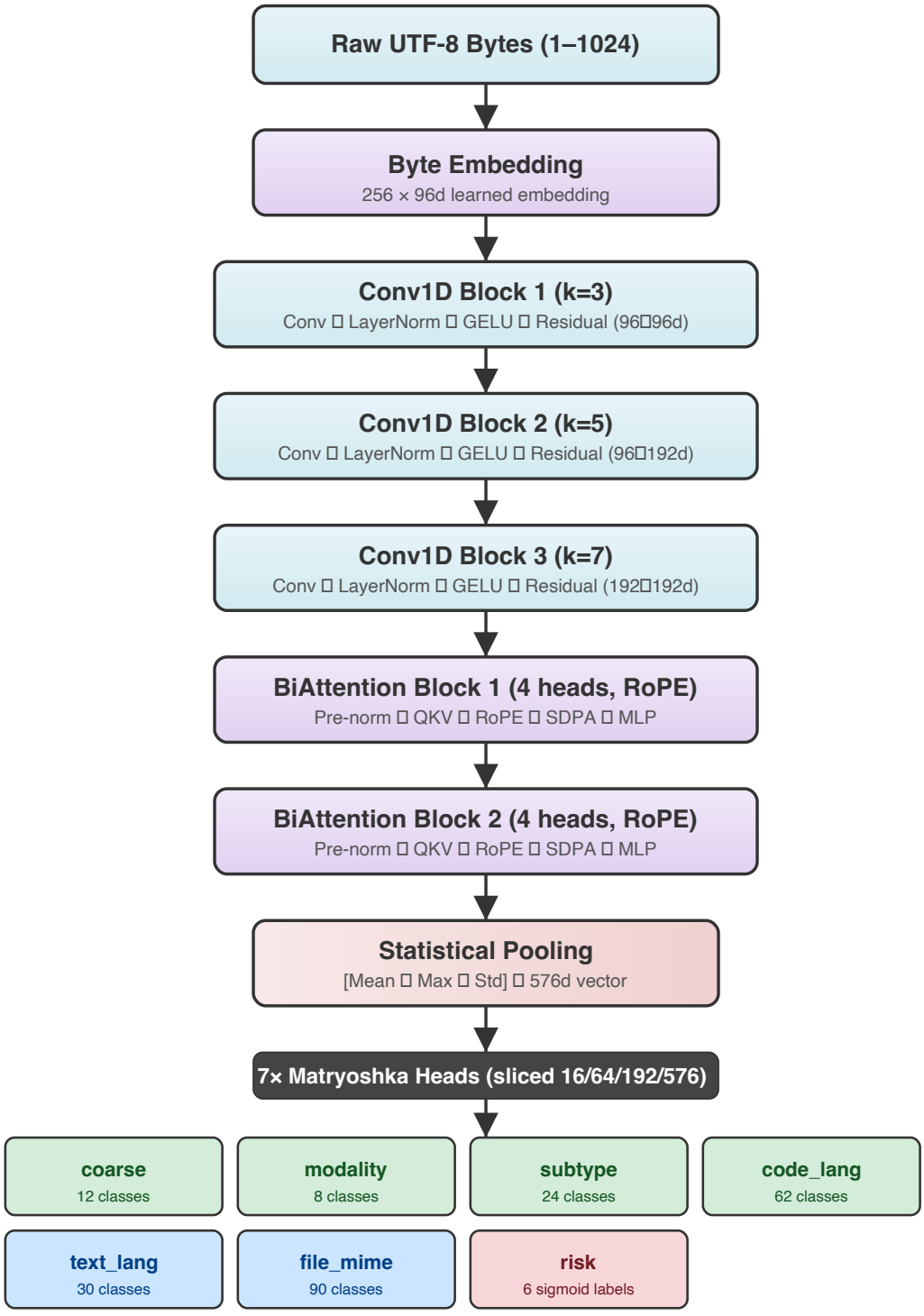

Figure 1: pico-type architecture. Raw bytes enter at the top and flow through embedding, three Conv1D blocks, two BiAttention blocks, and statistical pooling to produce a 576-dimensional shared representation. Seven Matryoshka heads slice this vector at four dimensions (16/64/192/576) for tiered classification.

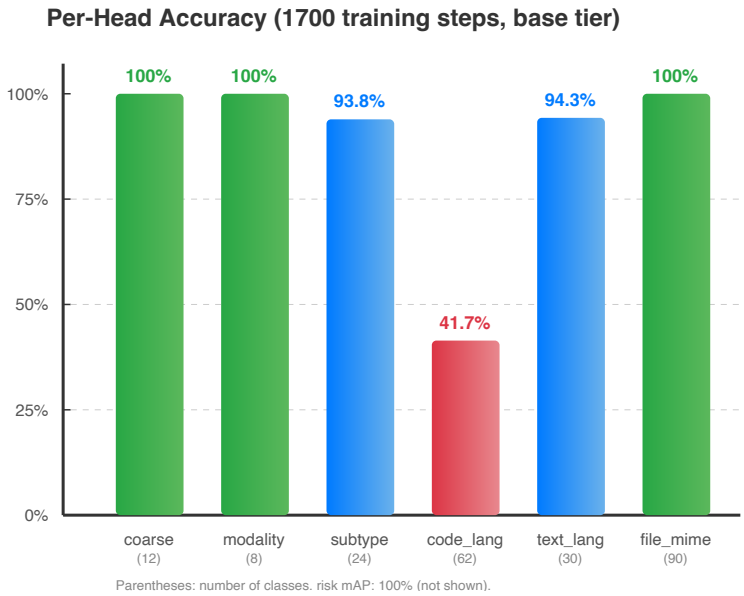

Figure 2: Per-head accuracy on 500 synthetic evaluation samples. Bar labels show accuracy and parenthesized class count. risk mAP of 100.0% is omitted.